

# DevOps: A Few Best Practices for Data Science Projects

DevOps practices play a vital role in deploying data models efficiently in production environments. This article outlines four excellent DevOps practices.

**D**evOps comprises the best practices to integrate software development and IT operations. The main aim is to reduce the development life cycle and to deliver products with high quality. Different sets of tools are used by people who practice DevOps. These sets of tools are known as toolchains, and are expected to fit into one or more categories of software development and delivery such as coding, building, testing, packaging, releasing, configuring and monitoring.

The aim of data science is to build data models and deploy them in production environments. DevOps plays a vital role in the data science process in order to build flexible and agile teams for efficient deployment of models. Let's briefly look at four DevOps best practices.



## User version control

In data science projects, keeping track of changes in code is very essential, as it involves multiple files and complex code. DevOps tools like Git play an important role in source code management. Git is a free and open source tool capable of handling any large data science project. It is a great tool to track source code changes, enabling multiple developers to work together in non-linear development. Linus Torvalds developed it in 2005 for the Linux operating system. It contains powerful features for version

control. Some of its important features are:

- History tracking
- Supports non-linear development
- Distributed development
- Supports collaboration
- Scalable in nature

Git is very well suited to data science projects as it supports distributed development and collaboration. Scalability is also required for these projects. The use of such a tool in a data science project ensures faster delivery and

improved quality. The workflow of Git is divided into three main states:

- **Working directory:** This is responsible for modifying files in the current working directory.
- **Index:** This is responsible for staging the files and adding a snapshot to a staging area.
- **Repository:** This performs a commit and saves the snapshot permanently in the Git directory. It is also responsible for checking the existing version and making the necessary changes.